CIS 4190/5190: Applied Machine Learning Fall 2025

CIS 5190 Final Project

Team Members: Rushil Patel, Sarthak Jain, and Tanner Shah *Project: [News Source]*

1 Introduction

In this project we investigated how to perform classification on news headlines. We experimented with a range of models (from simple statistical baselines to modern transformers). We found that out of them all DistilBERT performed the best, but lightweight transformers and LSTMs were not far behind (and much quicker). Once having identified the best model, we turned to seeing how well political sway in news headlines can be identified by finetuning our BERT model on a dataset of similarly polarized news outlets and tracking performance. We found that it worked surprisingly well, and it seemed to perform similarly if not better than current techniques that assess political sway.

2 Core Components

2.1 Data Collection and Dataset:

Initially, we scraped headlines from url-only-data.csv because we believed that was the correct source. That scraping required 2000 requests to each server, so to avoid detection and throttling I randomized request headers and added a small random jitter that staggered each request by 5 seconds. Because the process took a very long time, we ran the jobs on Penn Med HPC. Although we ultimately did not use those initial headline scrapes (project guidelines changed), the experience taught general large-scale scraping concepts that we reused for the sitemap-based URL collection described below. Dataset Here

For the actual dataset collection we switched to sitemap.xml scraping. We again used randomized headers and random jitter to avoid rate limiting, but replaced the slow per-request approach with a concurrent asyncio pipeline that retrieves sitemaps and article URLs much faster without actually scraping a headline. The sitemap pipeline is a two-step process:

- **Step 1 (fast, concurrent pass)**: request a large number of subsitemaps concurrently and extract article URLs via regex patterns (this avoids unnecessary full-page parsing and reduces load).
- **Step 2 (targeted retries)** : revisit any failed subsitemap requests using low-concurrency, staggered retries with multiple headers to ensure completeness.

We used regex-driven extraction, because many sitemaps and URL structures are highly regular (e.g., .../page=articles patterns), and BeautifulSoup-style full-page parsing would have caused excessive unnecessary requests and more rate-limiting. Concurrency + targeted retries allowed us to collect much larger datasets in practical time.

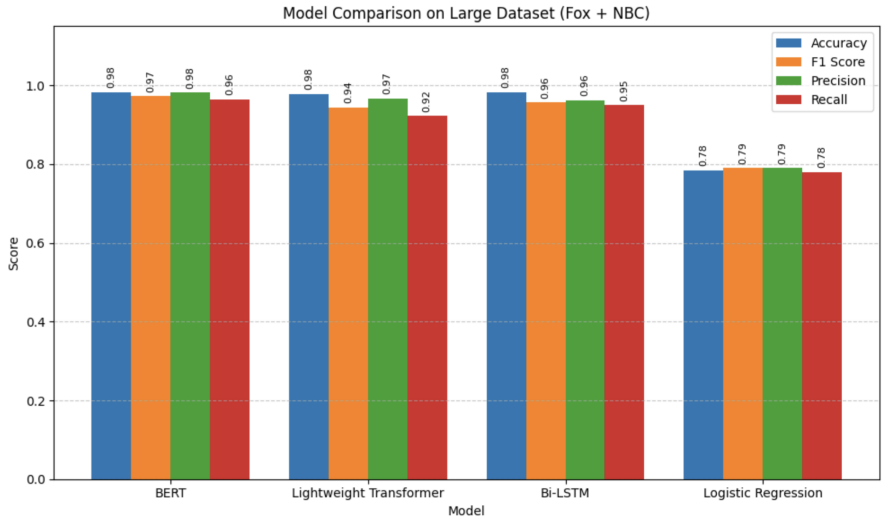
For specific sources (Fox, NBC, CNN, NPR, DailyWire), we located each site's master sitemap.xml (or sub-sitemaps) via simple web searches and followed the two-step process above. After collection, we performed standard cleaning: deduplication, removal of non-article URLs (index pages, tag pages, author pages), basic normalization (lowercasing, stripping excess at edn), and simple filtering to remove obviously malformed entries. We assigned labels by source (e.g., all DailyWire URLs labeled DailyWire) and recorded metadata such as publication date when available. The final dataset sizes varied by source (we had million URLs for Fox/NBC back each when available; smaller samples for CNN/DailyWire/NPR in some cases), and we tracked class balance carefully since imbalance affects evaluation and training choices. Getting a large dataset was an explicit priority because NLP models, especially Transformers, benefit strongly from more training data. For our training, we did 80/20 test/train split for training our models.

2.2 Model Design

To classify news article headlines into Fox News or NBC News sources, we evaluated five distinct architectures, ranging from traditional probabilistic models to state-of-the-art transformers. We began with baseline models, specifically Naive Bayes and Logistic Regression. These models are computationally efficient and offer rapid training times, making them excellent starting points. However, they rely fundamentally on a "bag-of-words" approach, treating each word as an independent feature. While Logistic Regression generally outperformed Naive Bayes by establishing linear decision boundaries, both models failed to capture deeper semantic context or word order. Consequently, they struggled with headlines where the sentiment or meaning depended heavily on phrasing rather than just the presence of specific keywords.

To address these limitations, we implemented a Long Short-Term Memory (LSTM) network. Unlike the static approaches of the baselines, the LSTM processes text as a sequence, allowing it to "remember" previous words and understand word order. This architecture provided a noticeable improvement in accuracy because it could interpret phrases rather than just isolated words. However, because standard LSTMs process text strictly from left to right, they often miss the full bidirectional context required to grasp the nuance of short, punchy political headlines.

Moving into transformer architectures, we first experimented with a Lightweight Transformer (ELECTRA Small). Unlike standard transformers that mask words, ELECTRA is trained as a discriminator to detect replaced tokens, offering a highly efficient solution with a very small parameter count (14 million). While effective as a lightweight option, our final and most successful model was DistilBERT. As a distilled version of the powerful BERT model, DistilBERT utilizes self-attention mechanisms to analyze every word in relation to every other word simultaneously. It provided the most robust "reading comprehension" of the headlines, capturing sarcasm and subtle framing differences more effectively than any other architecture we tested, making it our definitive choice for this classification task.



2.3 Evaluation and Model Performance

We evaluated models across internal train/test splits. Standard metrics used were accuracy, precision, recall, and F1 (we report per-class metrics and macro / weighted averages where relevant). For models trained on different dataset variants (e.g., "small dataset" vs. large dataset), we plotted comparative accuracy across model families to guide model selection for leaderboard submissions. How we chose models for leaderboard submission: we prioritized models that performed best all round. The figure below plots the statistics we tracked for BERT, the lightweight transformer, the LSTM, and logistic regression.

3 Exploratory Components

3.1 Motivation and Research Question

Motivation: The Computational Social Science Lab at Penn runs the Penn Media Accountability Project (PennMAP), which develops tools like the Media Bias Detector to track and classify top stories by political lean using AI and human raters. That project motivated our attempt to use ML to detect political bias from headlines/URLs. The central research question is:

Can we predict political bias from a news URL/headline?

Our hypothesis is that headline-level framing and word choice often contain sufficient signal to infer political slant or source identity; detecting source identity is a proxy for bias detection when sources have known leanings.

3.2 Method/Experimental Design

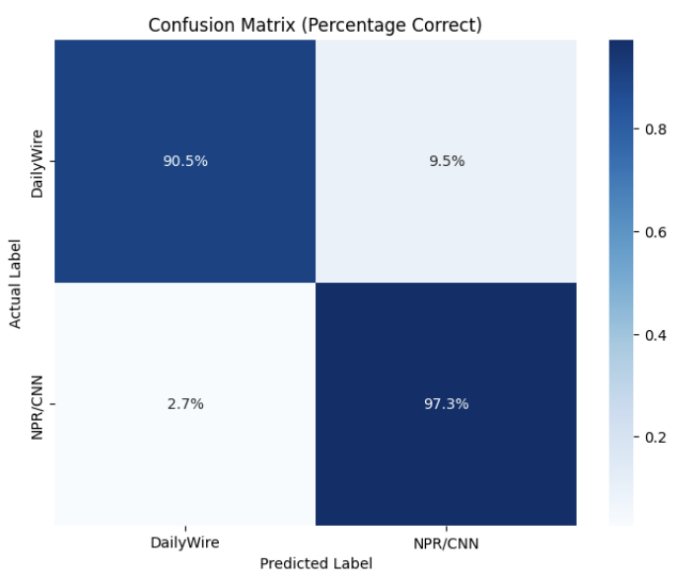
We evaluated the performance of the BERT Fox and NBC headline classifier on other news networks with a similar magnitude of political split: DailyWire versus CNN and NPR. The idea is that if the model performs well on both sets of outlets, this is indicative of some

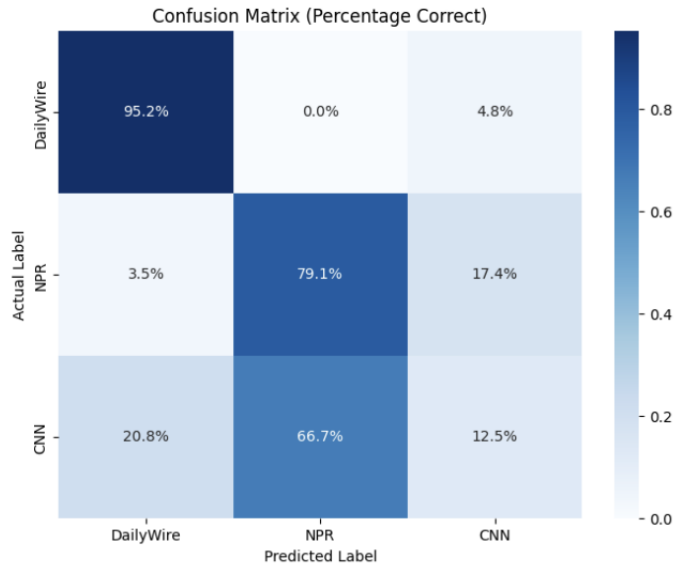
form of implicit political bias present in news headlines. To do so, our DistilBERT model was frozen except for the last two layers and the classifier head. The model was then fine-tuned on a dataset consisting of DailyWire, CNN, and NPR headlines, where DailyWire was assigned label "0" and CNN and NPR were assigned label "1" (conservative vs. left-leaning), and we report the corresponding training statistics. After this, we replaced the two-head classifier with a three-head classifier and fine-tuned the model on a dataset where DailyWire was assigned label "0," NPR label "1," and CNN label "2," in order to evaluate differences in political bias between NPR and CNN.

In order to use data from different news outlets other than FOX and NBC, we had to collect a new dataset. Using the same methodology used to collect our large FOX and NBC dataset, we collected around 400 NPR articles, 100 Dailywire articles, and 100 CNN articles respectively. We upsampled dailywire and/or CNN accordingly to account for this difference

3.3 results

Surprisingly, we had very good performance on the new dataset. When using a 2-head classifier (Dailywire v.s. NPR/CNN), we had accuracy $> .95$. The confusion matrix for the 3-head classifier is on the next page. As you can see, the model is very good at identifying DailyWire articles while tends to mix up NPR and CNN, supporting our hypothesis that political sway is present in headlines.





3.4 Comparison to SOTA Method

We compare our methodology to the PennMAP Media Bias Estimator, which measures political bias by using GPT models to label each article and its title for political "lean" on a numeric scale ranging from –5 (strongly Democratic) to +5 (strongly Republican), based on how the content aligns with party viewpoints, policies, or politicians.

To directly compare against this approach, we gave ChatGPT a prompt similar to the one used by PennMAP during training. Specifically, we provided 100 labeled news headlines from NPR and DailyWire as training examples, and then asked the model to label a new set of 100 unseen headlines consisting of 50 DailyWire and 50 NPR examples. The prompt instructed the model to act as an expert political content analyst, to focus on framing, word choice, implied concerns, and issue selection rather than publisher identity, and to assign each headline to one of two classes (DailyWire or NPR) while also providing a short textual justification for its decision in a strict JSON format.

Using NPR as the positive class and DailyWire as the negative class, the resulting evaluation showed 28 true positives, 0 false negatives, 10 false positives, and 39 true negatives, corresponding to an accuracy of 0.8701, a precision of 0.7368, a recall of 1.0000, and an F1 score of 0.848 for NPR. On this task, our fine-tuned model outperformed the ChatGPT-based approach, which we treated as a pseudo-state-of-the-art baseline for comparison.

3.5 Notes

- **takeaway:** Headline-level sentiment seems to be sufficient to identify source and detect partisan framing, especially for strongly partisan outlets.
- **Caveats:** Small sample sizes for some outlets (CNN, DailyWire in parts of our experiments). Also, we fine-tuned BERT on 3-4x more data than what we gave GPT. We also only compared two outlets. PennMAP's approach is also not tailored to this and is more sophisticated than what we emulated
- **Improvements:** Collect more balanced data across a wider set of outlets (left, center, right), consider continuous lean labeling instead of discrete classes, include article-body context or sentence-level labels where possible, and incorporate ongoing recalibration to manage temporal drift.

4 Team Contributions

- **Rushil:** Data collection/scraping/preprocessing for all 3 datasets (small FOX/NBC provided, large FOX/NBC, small Dailywire/CNN/NPR) and plots
- **Sarthak:** Model design and submission: Naive Bayes, transformer models, LSTM
- **Tanner:** Running DistilBERT, Exploratory Component, Report (Rushil and Sarthak wrote sections 2.1 and 2.2 respectively)